TASK 07 – From Procedural to Object-Oriented Programming

1. Aim

To compare procedural and object-oriented programming approaches by implementing metro fare calculations and smart card balance management using separate functions and a SmartCard class.

2. Procedural Programming

Procedural programming uses separate variables and functions to perform operations. The smart card balance is maintained separately and passed to functions for validation and fare deduction.

3. Object-Oriented Programming

Object-oriented programming combines data and functions into a class. The SmartCard class keeps the balance private and provides controlled access through canTravel() and deduct() methods.

4. Comparison of Procedural and Object-Oriented Approaches

Feature	**Procedural Approach**	**Object-Oriented Approach**
Structure	Separate functions	SmartCard class
Balance	Separate variable	Private data member
Data security	Less protection	Encapsulation

Balance deduction	Free function	deduct() method
Validation	Free function	canTravel() method
Maintainability	Changes may affect multiple functions	Centralized management

5. Balance Protection

The SmartCard class prevents negative or insufficient balances by checking whether the remaining balance is at least ₹50 before deducting the fare.

6. Minimum Balance Modification

In the procedural approach, the minimum balance condition may need to be updated in multiple functions. In the object-oriented approach, it can be maintained as a class constant and changed centrally.

7. Alternative Concept and Complexity Analysis

Parameter	**Private Balance**	**Public Balance**
Data access	Controlled through methods	Direct access
Time complexity	O(1)	O(1)
Space complexity	O(1)	O(1)

Balance protection	Enforced by class methods	Not guaranteed
Reliability	Higher	Lower

If the balance is public, any part of the program can directly modify it, potentially creating a negative balance or violating the minimum balance rule.

Alternative Concept: Public Balance

If the balance were declared as a public member, it could be accessed and modified directly.

Example:

```
class SmartCard

{

public:

   long long balance;

};
```

Another part of the program could execute:

```
card.balance = -5000;
```

This violates the smart card's balance protection rule because the balance can become negative without validation.

Complexity Analysis

Operation	Procedural	OOP
Fare calculation	O(1)	O(1)
Balance validation	O(1)	O(1)
Fare deduction	O(1)	O(1)
Additional space	O(1)	O(1)
Data protection	Limited	Stronger

Result: Both procedural and object-oriented implementations successfully calculate metro fares and process smart card transactions. The OOP implementation provides better encapsulation by protecting the balance through private data members and controlled methods.

Conclusion: Object-oriented programming provides better data protection, encapsulation and maintainability. Therefore, the SmartCard class approach is more suitable for a reliable metro fare management system.